



Ejercicio 1

```
def lee_luces(ruta_fichero: str) -> list[Iluminacion]:
    res = []
    with open(ruta_fichero, encoding="utf-8") as f:
        lector = csv.reader(f, delimiter=";")
        next(lector)
        for distrito, calle, tipo, fecha_inicio, fecha_fin, coste_base, \
            luminarias in lector:
            fecha_inicio = parsea_fecha(fecha_inicio)
            fecha_fin = parsea_fecha(fecha_fin)
            coste_base = float(coste_base)
            luminarias = parsea_luminarias(luminarias)
            res.append(Iluminacion(distrito, calle, tipo, fecha_inicio, \
                                    fecha_fin, coste_base, luminarias))

    return res

def parsea_fecha(fecha_str: str) -> date:
    return datetime.strptime(fecha_str, "%Y-%m-%d").date()

def parsea_luminarias(luminarias_str: str) -> list[Luminaria]:
    trozos = luminarias_str.split("*")
    return [parsea_luminaria(trozo) for trozo in trozos]

def parsea_luminaria(luminaria_str: str) -> Luminaria:
    nombre, cantidad, potencia = luminaria_str.split("-")
    return Luminaria(nombre.strip(), int(cantidad), float(potencia))
```

Test

```
def mostrar_iterable(iterable: Iterable) -> None:
    for it in iterable:
        print(f"\t{it}")

def test_lee_luces(luces: list[Iluminacion]) -> None:
    print("TEST lee_luces")
    print(f"Número de registros leídos:{len(luces)}")
    print("Los tres primeros")
    mostrar_iterable(luces[:3])
    print("Los tres últimos")
    mostrar_iterable(luces[-3:])

def main():
    luces = lee_luces(r"data\luces.csv")
    test_lee_luces(luces)

if __name__=="__main__":
    main()
```



Ejercicio 2

```
def potencia_total_de_distrito(luces: list[Iluminacion], distrito: str, \
    fecha_inicio: date|None = None) -> float:
    return sum (potencia_total(ilu_calle) for ilu_calle in luces\
        if ilu_calle.distrito == distrito and \
            (fecha_inicio is None or ilu_calle.fecha_inicio >= fecha_inicio))

def potencia_total(iluminacion: Iluminacion)->float:
    return sum(l.cantidad*l.potencia_w for l in iluminacion.luminarias)

## Solución con bucles
def potencia_total_de_distrito2(luces: list[Iluminacion], distrito: str, \
    fecha_inicio: date|None = None) -> float:
    total = 0.0
    for ilu_calle in luces:
        if ilu_calle.distrito == distrito and \
            (fecha_inicio is None or ilu_calle.fecha_inicio >= fecha_inicio):
            for luminaria in ilu_calle.luminarias:
                total += (luminaria.cantidad * luminaria.potencia_w)
    return total
```

Test

```
def test_potencia_total_de_distrito(luces: list[Iluminacion], distrito: str, \
    fecha_inicio: date|None = None) -> None:
    print(f"TEST potencia_total_de_distrito para distrito {distrito} desde {fecha_inicio}")
    res = potencia_total_de_distrito(luces, distrito, fecha_inicio)
    print(f"Potencia total: {res} W")

def main():
    luces = lee_luces(r"data\luces.csv")
    test_potencia_total_de_distrito(luces, "Bellavista", date(2024, 12, 1))
    test_potencia_total_de_distrito(luces, "Triana")

if __name__=="__main__":
    main()
```

Ejercicio 3

```
def distritos_con_todas_las_luminarias(luces: list[Iluminacion], nombres_luminarias: \
    set[str]) -> set[str]:
    d = luminarias_por_distrito(luces)
    return {distrito for distrito, luminarias in d.items()
        if nombres_luminarias.issubset(luminarias)
        # También nombres_luminarias < luminarias
    }
```



```
def luminarias_por_distrito(luces: list[Iluminacion]) -> dict[str, set[str]]:
    d = defaultdict(set)
    for i in luces:
        for luminaria in i.luminarias:
            d[i.distrito].add(luminaria.nombre)
    return d
```

Test

```
def test_distritos_con_todas_las_luminarias(luces: list[Iluminacion], \
        nombres_luminarias: set[str]) -> None:
    print(f"TEST distritos_con_todas_las_luminarias para luminarias \
{nombres_luminarias}")
    res = distritos_con_todas_las_luminarias(luces, nombres_luminarias)
    print(f"Distritos con todas las luminarias {res}")

def main():
    luces = lee_luces(r"data\luces.csv")
    test_distritos_con_todas_las_luminarias(luces,
        {'Guirnalda', 'Bola_Navideña', 'Estrella_Polar'})

if __name__ == "__main__":
    main()
```

Ejercicio 4

```
def distrito_mas_inversion(luces: list[Iluminacion], tipo_iluminacion: \
    str|None = None) -> tuple[str, float]:
    d = acumular_por_distrito(luces, tipo_iluminacion)
    return max(d.items(), key=lambda it:it[1])

def acumular_por_distrito(luces: list[Iluminacion], tipo_iluminacion: \
    str|None = None) -> dict[str, float]:
    d = defaultdict(float)
    for i in luces:
        if tipo_iluminacion is None or i.tipo == tipo_iluminacion:
            d[i.distrito] += i.coste_base
    return d
```

Test

```
def test_distrito_mas_inversion(luces: list[Iluminacion], tipo_iluminacion: \
    str|None = None) -> None:
    print(f"TEST distrito_mas_inversion para tipo iluminación {tipo_iluminacion}")
    distrito, inversion = distrito_mas_inversion(luces, tipo_iluminacion)
    print(f"Distrito con más inversión: {distrito} (Inversión: {inversion} €)" )
```



```
def main():
    luces = lee_luces(r"data\luces.csv")
    test_distrito_mas_inversion(luces, None)
    test_distrito_mas_inversion(luces, 'Proyeccion')

if __name__=="__main__":
    main()
```

Ejercicio 5

```
def ranking_variedad_normalizado_por_coste(luces: list[Iluminacion], \
    tipo_iluminacion: str, n: int = 3) -> list[str]:
    il_filtradas = [il for il in luces if il.tipo == tipo_iluminacion]
    il_ordenadas = sorted(il_filtradas, \
        key=lambda c: len(c.luminarias)/gasto_medio_distrito(c, il_filtradas),
        reverse=True)
    return [il.calle for il in il_ordenadas[:n]]

def gasto_medio_distrito(c: Iluminacion, luces: list[Iluminacion]) -> float:
    """Calcula el gasto medio por calle en un distrito dado"""
    il_distrito = [il for il in luces if il.calle.distrito == c.distrito]
    total_gasto = sum(il.coste_base for il in il_distrito)
    return total_gasto / len(il_distrito) if il_distrito else 0.0

##Solución 2:
def ranking_variedad_normalizado_por_coste2(luces: list[Iluminacion], \
    tipo_iluminacion:str, n: int = 3) -> list[str]:
    # Para ahorrar calculos, diccionario de coste medio por distrito
    d= media_coste_por_distrito(luces, tipo_iluminacion)
    gen = ((i.calle, indice_eficiencia(i, d))
        for i in luces
        if i.tipo == tipo_iluminacion)
    ord = sorted (gen, key=lambda t:t[1], reverse=True)[:n]
    return [calle for calle, _ in ord]

def indice_eficiencia(iluminacion: Iluminacion, media_coste: dict[str, float]) -> float:
    num_luminarias = sum(l.cantidad for l in iluminacion.luminarias)
    num_luminarias = len(iluminacion.luminarias)
    return num_luminarias/media_coste[iluminacion.distrito]

def media_coste_por_distrito(luces: list[Iluminacion], tipo_iluminacion: str):
    d = agrupar_por_distrito(luces, tipo_iluminacion)
    return {distrito: statistics.mean(i.coste_base for i in luces) \
        for distrito, luces in d.items()}

def agrupar_por_distrito(luces: list[Iluminacion], tipo_iluminacion: str|None = None) \
    -> dict[str, list[Iluminacion]]:
```



```
d = defaultdict(list)
for i in luces:
    if tipo_iluminacion is None or i.tipo == tipo_iluminacion:
        d[i.districto].append(i)
return d
```

Test

```
def test_ranking_variedad_normalizado_por_coste(luces: list[Iluminacion], \
        tipo_iluminacion: str, n: int = 3) -> None:
    print(f"TEST ranking_variedad_normalizado_por_coste para tipo '{tipo_iluminacion}' y n={n}")
    ranking = ranking_variedad_normalizado_por_coste(luces, tipo_iluminacion, n)
    print(f"Ranking de calles: {ranking}")

def main():
    luces = lee_luces(r"data\luces.csv")
    test_ranking_variedad_normalizado_por_coste(luces, 'LED_Dinamico', 5)

if __name__ == "__main__":
    main()
```

Ejercicio 6

```
def coste_medio_diario_por_distrito(luces: list[Iluminacion], \
        umbral_coste: float = 100.0) -> list[tuple[str, float]]:
    resultado = {}
    distritos = agrupar_por_distrito(luces)
    for distrito, iluminaciones in distritos.items():
        coste_diario = coste_medio_diario(iluminaciones)
        if coste_diario > umbral_coste:
            resultado[distrito] = coste_diario
    return resultado

def coste_medio_diario(luces: list[Iluminacion]) -> float:
    total_coste = 0.0
    total_dias = 0
    for iluminacion in luces:
        dias_activas = (iluminacion.fecha_fin - iluminacion.fecha_inicio).days
        total_coste += iluminacion.coste_base
        total_dias += dias_activas
    res = 0
    if total_dias > 0:
        res = total_coste / total_dias
    return res
```



Test

```
def test_coste_medio_diario_por_distrito(luces: list[Iluminacion], \
    umbral_coste: float = 100.0) -> list[tuple[str, float]]:
    print(f"TEST coste_promedio_por_distrito para umbral{umbral_coste}")
    res = coste_medio_diario_por_distrito(luces, umbral_coste)
    for distrito, coste in res.items():
        print(f"distrito: {distrito}, Coste medio diario: {coste:.2f} €/día")

def main():
    luces = lee_luces(r"data\luces.csv")
    test_coste_medio_diario_por_distrito(luces)
    test_coste_medio_diario_por_distrito(luces, 300)

if __name__ == "__main__":
    main()
```